COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S1
DATE PERFORMED:	8/18/26	**DATE SUBMITTED:**	8/18/26
NAME:	Vince Joseph Garcia	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. __6.1_ Searching Techniques

1. PROCEDURE OUTPUT

Node.h

```h
#ifndef NODE_H
#define NODE_H

template <typename T>
class Node{
    public:
        T data;
        Node<T>* next;
};

template <typename T>
Node<T>* createNode(T newData){
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

#endif
```

SearchingAlgo.h

```
#include <iostream>
#include "node.h"

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void linearSearchArr(T data[], int arrSize, T datafind){

    int i=0;

    while(i < arrSize){
        if(datafind == data[i]){
        std::cout<<"the search is success"<<std::endl;
        return;
        }
        i++;
    }

    std::cout<<"the element is not found"<<std::endl;
    return;
```

```
22
23  }
24  template <typename T>
25  void linearSearchLL(Node<T>* node, T datafind){
26      while(node != nullptr){
27          if(datafind == node->data){
28              std::cout<<"the search is success"<<std::endl;
29              return;
30          }
31          node = node->next;
32      }
33      std::cout<<"the search was unsuccessful"<<std::endl;
34      return;
35
36  }
37  template <typename T>
38  void linearsearchLL(Node<T>* node, T datafind){
39      while(node != nullptr){
40          if(node->data == datafind){
41              std::cout<<"the search is success"<<std::endl;
42              return;
43          }
```

```
44          node = node->next;
45      }
46      std::cout<<"the search was unsuccessful"<<std::endl;
47  }
48
49  template <typename T>
50  Node<T>* getMiddle(Node<T>* start, Node<T>* last){
51      Node<T>* slow = start;
52      Node<T>* fast = start;
53      while(fast != last && fast->next != last){
54          slow = slow->next;
55          fast = fast->next->next;
56      }
57      return slow;
58  }
59  template <typename T>
60  void binarySearchLL(Node<T>* head, T datafind){
61      Node<T>* start = head;
62      Node<T>* end = nullptr;
63
64      while(start != end){
65          Node<T>* mid = getMiddle<T>(start, end);
```

```
64      while(start != end){
65          Node<T>* mid = getMiddle<T>(start, end);
66          if(mid == nullptr) break;
67          if(mid->data == datafind){
68              std::cout<<"the search is success"<<std::endl;
69              return;
70          }
71          if(mid->data < datafind){
72              start = mid->next;
73          } else {
74              end = mid;
75          }
76      }
77
78      std::cout<<"the element is not found"<<std::endl;
79      return;
80
81  }
82  //create a binary search using array : getMiddle and / binarySearchArr
83
84
85  #endif
86
```

MainCpp

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){

    int dataset[max_size];
    srand(time(0)); // Seed the random number generator with the current time
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand();
    }
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

    int toFind;
    std::cout<<"what value you want to search: ";
    std::cin>>toFind;


```

```
26      linearSearchArr(dataset, max_size, toFind);
27
28      Node<char>* name1 = createNode('J');
29      Node<char>* name2 = createNode('I');
30      Node<char>* name3 = createNode('M');
31      Node<char>* name4 = createNode('L');
32      Node<char>* name5 = createNode('O');
33      Node<char>* name6 = createNode('R');
34      Node<char>* name7 = createNode('D');
35
36      name1->next = name2;
37      name2->next = name3;
38      name3->next = name4;
39      name4->next = name5;
40      name5->next = name6;
41      name6->next = name7;
42      name7->next = nullptr;
43
44      std::cout<<"Linear search for linked list result: ";
45      linearSearchLL(name1, 'D');
46      std::cout<<"Linear search for linked list result: ";
47      linearSearchLL(name1, 'x');
48
49      std::cout<<"Binary search for linked list result: ";
50      binarySearchLL<char>(name1, 'M');
51      std::cout<<"Binary search for linked list result: ";
52      binarySearchLL<char>(name1, 'x');
53
54
55      return 0;
56  }
57
```

Code Output

```
2974 2229 8811 19400 12264 22478 17530 28112 3694 22182 16374 15756 23019 16474 10655 27274 31306 14711 16087 31603 24344 10407 27571 3033 22074 656 30546 5935
 15067 3321 19053 29977 4265 23854 1072 27367 24385 5542 20808 30299 25063 9956 14654 13747 24443 6043 22019 4323 16040 6165
what value you want to search: 1072
the search is success
Linear search for linked list result: the search is success
Linear search for linked list result: the search was unsuccessful
Binary search for linked list result: the element is not found
Binary search for linked list result: the element is not found
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.